

APPZETO MACHINE TEST

Full Stack Developer - Machine Test

Duration: 2 hours

Objective: Evaluate practical full-stack engineering across architecture, database design, authentication, authorization, concurrency, real-time systems, performance, security, testing, and production readiness.

Note: AI coding assistants may be used. Candidates must be able to explain and defend significant implementation decisions.

General Technical Requirements

- Frontend: React.js / Next.js. TypeScript is preferred.
- Backend: Node.js with Express.js, NestJS, or Fastify.
- Database: MongoDB or PostgreSQL.
- Redis, WebSockets, queues, Docker, and similar technologies may be used where appropriate.
- The application must be functional, modular, and maintainable.
- Validation must be performed on the backend as well as the frontend where applicable.
- Use appropriate HTTP status codes and meaningful API error responses.
- Never commit secrets or credentials to the repository.
- Use appropriate database indexes.
- Handle API and infrastructure failures gracefully.
- Each task must include a README covering setup, environment variables, architecture, database structure, API documentation, technical decisions, and known limitations.

TASK 1: Real-Time Auction & Bidding Platform

Overview

Build a web application for an online auction platform. Sellers list items for timed auctions, bidders place bids in real time, and the platform settles the winning bid when the auction closes. The system must keep the highest bid and each bidder's wallet balance correct when many users bid on the same item at the same moment.

1. Authentication

- Registration, login, and logout.
- Password hashing.
- Access-token authentication.
- Protected APIs.
- Roles: BIDDER, SELLER, and ADMIN.

2. Auction Management

- Sellers can create, update, and delete auctions while they are in DRAFT state.
- Configure title, description, images, category, starting price, minimum bid increment, optional reserve price, start time, and end time.
- Admins can approve/reject auctions before they go live and can force-cancel any auction.
- An auction cannot be edited once it has received its first bid.

3. Auction Lifecycle

```
DRAFT
  |
  v
SCHEDULED
  |
  v
LIVE -- end time reached --> CLOSED --> SETTLED
```

```
|
+---- admin cancel -----> CANCELLED
```

Transitions to LIVE and CLOSED must happen automatically at the configured times (use a scheduler or delayed job), not when a user happens to load the page. Invalid transitions must be rejected.

4. Wallet & Bid Holds

- Each bidder has a wallet with an **available** balance and a **held** balance. A mock top-up endpoint may be used.
- Placing a bid places a hold on the bid amount. When a bidder is outbid, their hold is released immediately.
- A bidder can never have holds exceeding their wallet balance, across all auctions combined.
- On settlement, the winner's hold is captured and credited to the seller; all other holds are released.

5. Bidding Rules

- A bid must be at least *current highest bid + minimum increment* (or the starting price for the first bid).
- Sellers cannot bid on their own auctions.
- **Anti-sniping:** if a valid bid is placed within the last 2 minutes, the end time is extended by 2 minutes (configurable). Clients must reflect the new end time.
- If the reserve price is not met at close, the auction closes with no winner and all holds are released.

6. Concurrency Requirement

The implementation must guarantee that the recorded highest bid is always correct and that no two bids can both succeed at the same amount. For example, if 100 bidders concurrently submit a bid of the same valid amount, exactly one must succeed and the rest must be rejected with a meaningful error. Wallet holds must never go negative or exceed the balance under concurrent bidding across multiple auctions.

Create an automated test demonstrating both behaviours and document the database/concurrency strategy used.

7. Real-Time Requirement

Use WebSockets, Socket.IO, or an equivalent real-time mechanism. All clients viewing an auction must see new bids, the current highest bid, the countdown (including anti-sniping extensions), and the final result without a page refresh. An outbid user must receive a real-time notification. Clients that reconnect must resynchronise state.

8. Admin Dashboard

- Total auctions by state.
- Live auctions.
- Total bids.
- Gross merchandise value (settled).
- Auctions closed without a winner.
- Cancelled auctions.
- Recent bids.
- At least one meaningful chart.

9. Required APIs

```
POST /api/auth/register
POST /api/auth/login
GET  /api/auctions
POST /api/auctions
GET  /api/auctions/:id
PUT  /api/auctions/:id
DELETE /api/auctions/:id
POST /api/auctions/:id/approve
POST /api/auctions/:id/cancel
GET  /api/auctions/:id/bids
POST /api/auctions/:id/bids
```

GET /api/wallet
POST /api/wallet/topup
GET /api/me/bids
GET /api/admin/dashboard

APPZETO MACHINE TEST

Full Stack Developer - Machine Test

TASK 2: Multi-Tenant Customer Support Helpdesk

Overview

Build a helpdesk SaaS application where companies manage customer support tickets. It must support multiple workspaces, teams, role-based permissions, SLA tracking, real-time agent collaboration, audit logs, search, and notifications.

1. Workspace Structure

```
User
+-- Workspace A
|   +-- Team: Billing
|   +-- Team: Technical
|
+-- Workspace B
    +-- Team: General
```

A user may belong to multiple workspaces. Workspace roles are OWNER, ADMIN, AGENT, and CUSTOMER. Data from one workspace must never be visible from another.

2. Tickets

- Customers can raise tickets; agents can also create tickets on behalf of customers.
- Tickets must support subject, description, status, priority, team, assignee, tags, attachments, public replies, internal notes (never visible to customers), and activity history.
- Each workspace has its own human-readable ticket number sequence (e.g. A-1001, A-1002) with no gaps or duplicates under concurrent creation.

3. Ticket Lifecycle

```
OPEN --> IN_PROGRESS --> WAITING_ON_CUSTOMER --> RESOLVED --> CLOSED
      ^               |               |
      +---- customer replies ----+   reopen within 7 days
```

Invalid transitions must be rejected server-side (e.g. CLOSED -> IN_PROGRESS).

4. Queue View & Assignment

- Agents see a queue view grouped by status with drag-and-drop between columns; changes must persist.
- Agents can "claim" an unassigned ticket. If two agents claim the same ticket simultaneously, exactly one must succeed.
- Optional auto-assignment: round-robin across available agents in the ticket's team.

5. SLA Tracking

- Each priority has a first-response target and a resolution target (e.g. URGENT: 1h / 8h).
- SLA timers pause while the ticket is WAITING_ON_CUSTOMER.
- A background job must detect tickets approaching and breaching SLA and mark them accordingly.

6. Real-Time Collaboration

When two agents are viewing the same ticket, new replies, notes, and status/assignment changes must appear without a page refresh. Show a "someone else is viewing/typing" indicator to prevent duplicate replies. Use WebSockets, Socket.IO, or an equivalent solution.

7. Permission System

Role	Permissions
CUSTOMER	Create tickets, view and reply to their own tickets only. Cannot see internal notes.

AGENT	View tickets of their teams, reply, add internal notes, claim and update tickets.
ADMIN	Manage teams, agents, SLA policies, reassign and delete tickets.
OWNER	Full workspace access, including billing settings and role changes.

Authorization must be enforced server-side. Do not rely on hiding or disabling frontend buttons.

8. Audit Log

Every important action must generate an audit entry containing:

user | workspace | action | entity | entityId | oldValue | newValue | timestamp

Examples include status changes, reassignment, priority changes, SLA policy edits, and role changes. Provide a searchable and filterable audit-log page for authorized users.

9. Advanced Search

Support combined filters for status, priority, team, assignee, tags, SLA state, created date, and full-text search across subject and messages.

```
status=OPEN
priority=URGENT
team=billing
slaState=BREACHED
createdAt>2026-09-01
q=refund
```

10. Notifications

- Notify agents when a ticket is assigned to them.
- Notify users when mentioned in an internal note.
- Notify customers when an agent replies.
- Notify agents and admins when an SLA is about to breach or breaches.
- Notify users when added to a workspace.
- Notifications should update in real time.

11. Performance Requirements

Assume the system may contain 500,000 tickets, 2,000,000 messages, 20,000 users, and 1,000 workspaces.

- Use appropriate database indexes.
- Implement cursor-based pagination for tickets and messages.
- Avoid fetching unnecessary records.
- Avoid N+1 queries.
- Debounce search where appropriate.
- Explain the indexing strategy in the README.

12. Required APIs

```
POST /api/workspaces
GET /api/workspaces
POST /api/workspaces/:id/members
DELETE /api/workspaces/:id/members/:userId
POST /api/teams
POST /api/tickets
GET /api/tickets
GET /api/tickets/:id
PATCH /api/tickets/:id
POST /api/tickets/:id/claim
DELETE /api/tickets/:id
POST /api/tickets/:id/messages
GET /api/audit-logs
GET /api/notifications
PATCH /api/notifications/:id/read
```

APPZETO MACHINE TEST

Full Stack Developer - Machine Test

TASK 3: Food Ordering & Delivery Dispatch System

Overview

Build a production-style food ordering application covering restaurant menus, carts, checkout, limited-quantity items, order processing, rider dispatch, idempotency, background jobs, payment webhooks, caching considerations, security, and observability.

1. Authentication

- Registration, login, logout, and profile management.
- Delivery address management.
- Roles: CUSTOMER, RESTAURANT_OWNER, RIDER, and ADMIN.

2. Restaurants & Menu

- Restaurants have name, cuisine, address, opening hours, delivery radius, and an open/closed toggle.
- Menu items have name, description, images, category, price, discount, availability, and SKU.
- Items support required and optional customisation groups with price modifiers.

Margherita Pizza

+- Size (required, pick 1): Regular +0 | Medium +120 | Large +220

+- Crust (required, pick 1): Classic +0 | Thin +0 | Cheese Burst +90

+- Toppings (optional, max 3): Olives +40 | Jalapeno +40 | Paneer +60

- Some items are **limited daily quantity** (e.g. only 30 "Chef's Special" per day).

3. Cart

- Add items with customisations.
- Remove items.
- Change quantity.
- View cart.
- Calculate totals.
- A cart may contain items from only one restaurant at a time.
- Cart must survive browser refresh.

4. Checkout

Calculate the following on the backend:

```
Item Subtotal (incl. customisations)
- Coupon Discount
+ Packaging Charges
+ Delivery Fee (distance-based)
+ Tax
-----
Final Total
```

Never trust prices sent from the frontend. Reject checkout if the restaurant is closed or the address is outside the delivery radius.

5. Inventory & Coupon Consistency

If a limited item has 5 portions left and 20 users simultaneously attempt to order one each, only 5 orders should successfully reserve a portion. Quantity must never become negative or oversold. Similarly, a coupon with a global usage limit of 100 must never be redeemed more than 100 times, and a single-use-per-user coupon must never be redeemed twice by the same user under concurrent requests.

Document how concurrent inventory and coupon updates are handled.

6. Order State Machine

```
PENDING
|
PAYMENT_PROCESSING
|
PLACED
|
ACCEPTED (restaurant)
|
PREPARING
|
READY_FOR_PICKUP
|
OUT_FOR_DELIVERY
|
DELIVERED
```

Failure states:
PAYMENT_FAILED
REJECTED (restaurant)
CANCELLED
REFUNDED

Invalid transitions must be rejected. For example, DELIVERED -> PREPARING must not be allowed. Only the correct actor may perform each transition (e.g. only the restaurant may ACCEPT; only the assigned rider may mark OUT_FOR_DELIVERY or DELIVERED).

7. Rider Dispatch

- Riders can go online/offline and update their location.
- When an order is READY_FOR_PICKUP, it is offered to nearby online riders.
- If multiple riders accept the same order concurrently, exactly one must be assigned.
- If no rider accepts within a configurable time, the offer is re-broadcast with a wider radius.

8. Payment Simulation

Create a mock payment provider supporting SUCCESS, FAILED, and TIMEOUT outcomes. The system must correctly handle each outcome, including releasing reserved inventory and coupon usage on failure.

9. Idempotency

Support an idempotency key for order and payment operations.

Idempotency-Key: abc123

If the same request is sent multiple times, the system must not create duplicate orders or duplicate payment effects. The same key reused with a different request body must be rejected.

10. Background Jobs

Use BullMQ, RabbitMQ, a Redis-based queue, or an equivalent technology.

- Order status emails/SMS (mocked).
- Auto-cancel and refund orders a restaurant does not accept within a configurable time.
- Automatic release of expired inventory reservations.
- Rider re-dispatch when an offer times out.
- Daily reset of limited-quantity items.

11. Real-Time Tracking

Customers must see order status changes and the rider's live location without a page refresh. Restaurants must receive new orders in real time.

12. Payment Webhooks

Create a mock payment webhook endpoint. The same webhook may be delivered multiple times and out of order. Processing must be idempotent, and webhook signatures must be verified.

POST /api/webhooks/payment

13. Admin Dashboard

- Total orders.
- Revenue.
- Active orders by state.
- Failed payments.
- Restaurant rejection rate.
- Average delivery time.
- Top restaurants and top items.
- Date filters.
- Order search.
- Restaurant search.
- Basic sales analytics.

14. Security Requirements

- Protect against SQL/NoSQL injection.
- Protect against XSS.
- Handle CSRF where applicable.
- Prevent broken authorization (e.g. a restaurant must not see another restaurant's orders; a rider must not update an order not assigned to them).
- Prevent mass assignment.
- Prevent sensitive information exposure.
- Apply rate limiting.
- Validate requests.
- Use authentication and authorization middleware.
- Use secure headers.
- Return safe, structured errors.

15. Required APIs

POST /api/auth/register
POST /api/auth/login
GET /api/restaurants
GET /api/restaurants/:id/menu
POST /api/restaurants/:id/menu-items
PUT /api/menu-items/:id
DELETE /api/menu-items/:id
GET /api/cart
POST /api/cart/items
PATCH /api/cart/items/:id
DELETE /api/cart/items/:id
POST /api/orders
GET /api/orders
GET /api/orders/:id
PATCH /api/orders/:id/status
POST /api/orders/:id/cancel
POST /api/riders/offers/:orderId/accept
POST /api/payments
POST /api/webhooks/payment
GET /api/admin/dashboard

APPZETO MACHINE TEST

Full Stack Developer - Machine Test

Testing Requirements

Each task must contain automated tests covering important business logic and API behavior.

Unit Tests

Test important business logic, validation, state transitions, calculations, and other isolated components.

Integration Tests

Test APIs together with the database and relevant infrastructure.

Edge Cases

- Invalid authentication.
- Unauthorized access.
- Expired tokens.
- Invalid input.
- Missing resources.
- Duplicate requests.
- Concurrent requests.
- Database failures.
- Payment failures.
- Scheduled job failures and retries.

Evaluation Criteria

Category	Weight
Architecture & Code Quality	15%
Backend Engineering	15%
Database Design	10%
Concurrency & Data Consistency	15%
Authentication & Authorization	10%
Frontend Quality & UX	10%
Real-Time Features	5%
Performance	5%
Security	5%
Testing	5%
Documentation	5%
Total	100%

Submission Requirements

Submit a Git repository containing a complete implementation of the assigned tasks.

```
project/  
+-- frontend/  
+-- backend/  
+-- tests/
```

```
+-- docker-compose.yml
+-- README.md
+-- .env.example
+-- ...
```

README Requirements

- How to run the project.
- Environment variables.
- Database setup.
- Architecture.
- API documentation.
- Database and indexing strategy.
- Concurrency strategy.
- Security considerations.
- Testing strategy.
- Known limitations.
- What you would improve with another 2 days.

Engineering Expectations

- Prioritize correctness and data consistency.
- Keep business logic out of presentation components.
- Use clear separation of concerns.
- Design APIs around clear resources and business operations.
- Handle failures and unexpected states explicitly.
- Write maintainable code rather than optimizing only for feature count.
- Document important trade-offs and assumptions.